

#UNIT-3

#Artificial Neural Network (ANN) -

- (i) Brain inspired parallel computing model
- (ii) Used for classification, pattern recognition, optimization, clustering
- (iii) Consists of interconnected neurons (nodes)
- (iv) Structure:

- Neurons (nodes): basic processing units
- Connections (links): connect neurons
- Weights: control strength of signals
- Parallel processing across neurons

(v) Working:

- Inputs $\rightarrow$ weighted sum $\rightarrow$ activation function $\rightarrow$ output
- Each neuron has an activation state (signal)
- Output passed to next neurons

(vi) Also called connectionist / distributed systems

(vii) Learns by adjusting weights

(viii) Weights can excite or inhibit signals

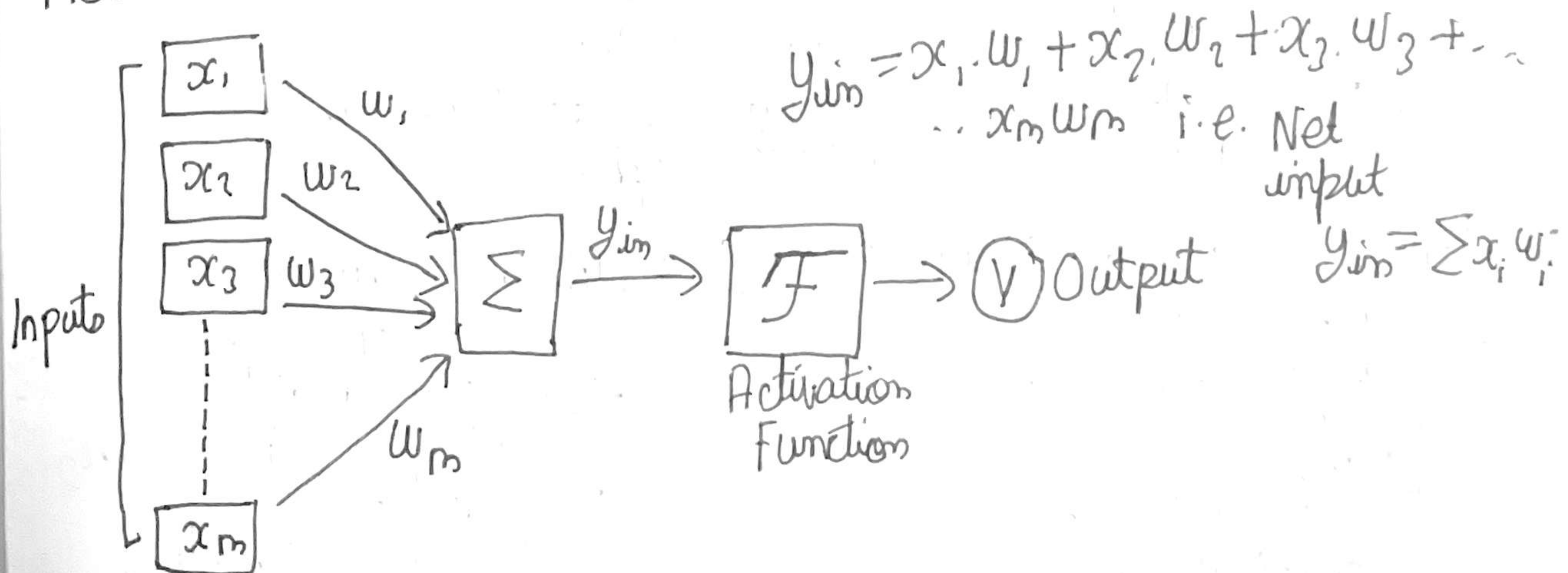
(ix) Inspired by human brain neurons

(x) Large no. of neurons + connections

- dendrites = receive signals
- Soma (cell body) = processes signals
- Axon = sends signals
- Synapse = connection between neurons

Q. BNN vs ANN

Model:



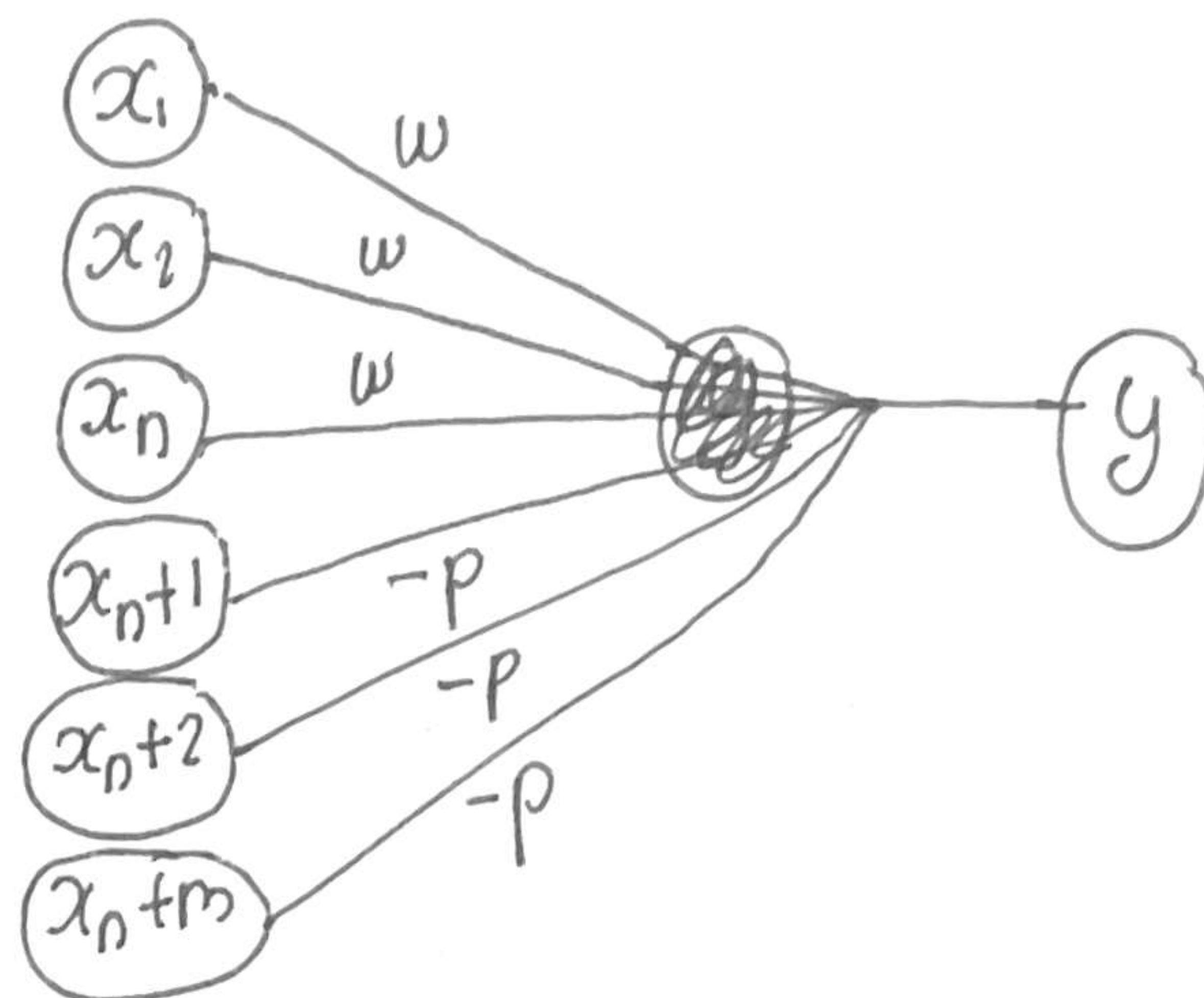
(xi) Output:

- Output: $y = F(y_{im})$ (activation on net input)
- Net input: weighted sum of inputs
- ANN mimics brain $\rightarrow$ learns patterns & makes predictions
- 13 layers of interconnected neurons

McCulloch Pitts Model -

- (i) Early neuron model (linear threshold unit)
- (ii) Inputs + weights $\rightarrow$ weighted sum
- (iii) Output rule:
 - $\begin{cases} \text{If sum} \geq \text{threshold} \rightarrow 1 \\ \text{Else} \rightarrow 0 \end{cases}$

- (iv) Formula: $y_{in} = \sum x_i w_i$
- (v) Limitation: works only for linearly separable problems



Perceptron (Single Layer) -

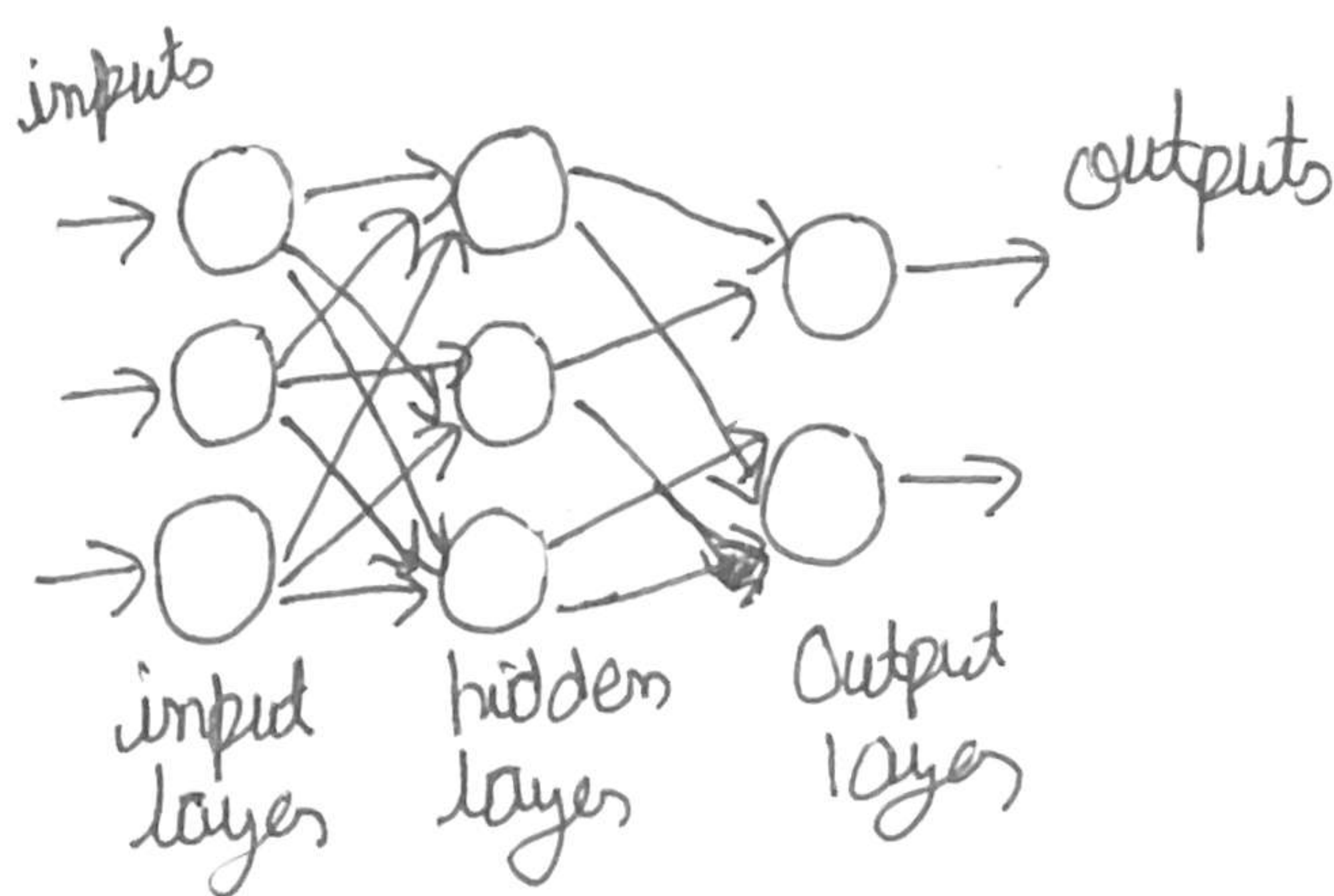
- (i) Input layer + output neuron
- (ii) Uses threshold function
- (iii) Suitable for binary classification
- (iv) Cannot solve non linear problems (eg. XOR)

$$y = \begin{cases} 1, & \text{if } \sum w_i I_i > t \\ 0, & \text{if } \sum w_i I_i < t \end{cases}$$

$t = \text{threshold}$

Multi Layer Perceptron (MLP) -

- (i) Input layer: takes data
- (ii) Hidden layers: extract features
- (iii) Output layer: gives prediction
- (iv) Solves complex/non linear problems



Activation Functions -

Function	Formula	Range
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	(0, 1)
Tanh	$\tanh(x) = 2\sigma(2x) - 1$	(-1, 1)
ReLU	$f(x) = \max(0, x)$	$\in [0, \infty)$

#ANN algorithm -

- (i) Initialize weights & bias
- (ii) Feed input data
- (iii) Forward propagation (calculate output) → uses activation function
- (iv) Calculate error
- (v) Backpropagation (update weights) → uses gradient descent $w = w - \alpha \frac{\partial \text{Error}}{\partial w}$
- (vi) Repeat (epochs)
- (vii) Test the network

~~#ANN~~ #Advantages -

- (i) Noise resilience
- (ii) Versatility
- (iii) Efficiency
- (iv) Parallel processing

#Limitations -

- (i) Overfitting
- (ii) Data dependency
- (iii) Computational expense

#Discrimination Power of Single Neuron -

- (i) Can distinguish subtle patterns
- (ii) Encodes info via firing rate (spikes/sec) or spike count
- (iii) How Neurons discriminate?
 - Spike rate/count (represents stimulus strength)
 - Temporal patterns (timing of spikes carries info)
 - Location/features (detect spatial patterns)
- (iv) Factors affecting power
 - Neural architecture: active dendrites ↑ capacity
 - Stimulus type: temporal (audio) vs spatial (visual)
 - Readout efficiency: downstream interpretation matters
 - Learning/context: improves neuron tuning
- (v) Limitations
 - Linear ~~sep~~ separability
 - Noise/variability
 - Complex tasks requires multiple neurons (networks)

Training Single Perceptron (Delta Rule) -

- (i) Based on gradient descent
- (ii) Goal: minimize error (target - output)
- (iii) Weight update: $\Delta w = \alpha \cdot x \cdot \text{error}$
- (iv) Applies to continuous activation

Weight Update Cases -
(a) If $t \neq y \rightarrow$ update weights
(b) If $t = y \rightarrow$ no change

Multilayer Neural Networks (MLP) -

- (i) Stacks of multiple layers of neurons
- (ii) Each layer applies activation function
- (iii) Learn complex, non linear relationship
- (iv) Uses backpropagation for training

Error Calculation -
Error (MSE)
 $E = \frac{1}{2} (t - y)^2$

Forward Propagation -

- (i) Compute weighted sum:
 - (a) $h_1 = w_1 x_1 + w_2 x_2 + b_1$
 - (b) $h_2 = w_3 x_1 + w_4 x_2 + b_2$
- (ii) Apply activation:
 - (a) $a(h_1), a(h_2)$
- (iii) Output:
 $y = w_5 \cdot a(h_1) + w_6 \cdot a(h_2) + b_3$
- (iv) Final Prediction: $a(y)$

Backpropagation -

- (i) Uses chain rule ~~for~~ to compute gradients
- (ii) Updates weights
 $\rightarrow w(\text{new}) = w - \eta \left(\frac{\partial E}{\partial w} \right)$
- (iii) Adjusts weights iteratively to reduce error

Key Idea -

- (i) Forward pass $\rightarrow$ compute output
- (ii) Backward pass $\rightarrow$ update weights
- (iii) Repeat $\rightarrow$ model learns & improves predictions

Activation Functions -

- 1) Decide neuron output (activate or not)
- 2) Applied on weighted sum of inputs
- 3) Introduce non linearity $\rightarrow$ learn complex patterns
- 4) What they do?
 - Transform input $\rightarrow$ output signal
 - Pass output to next layer
 - Without them $\rightarrow$ network = linear model only

5) Why?

- Enable learning of complex non linear relationships
- Allows deep feature extraction
- Essential for:

- Image classification
- Speech recognition
- NLP

6) Applications

- Image Recognition (ReLU & Softmax)
- NLP (Tanh & Softmax)
- Healthcare
- Fraud detection (Sigmoid)

Types of Activation Functions -

1) Linear Activation -

- Formula: $f(x) = ax$
- Output: scaled input
- Pros: Used in regression
- Con: No non linearity $\rightarrow$ limited learning

2) Sigmoid -

- Formula: $1/(1 + e^{-x})$
- Output: (0, 1)
- Pros: Interpretable as probability
- Con: Vanishing gradient, slow training

3) Tanh -

- Formula: $(e^x - e^{-x}) / (e^x + e^{-x})$
- Output: $(-1, 1)$
- Pros: Zero centered, better gradient than sigmoid
- Con: Still vanishing gradient descent

4) ReLU -

- Formula: $\max(0, x)$
- Pros: Fast, efficient avoids vanishing gradient
- Con: Dying ReLU problem (inactive neurons)
- Most used in deep learning

5) Leaky ReLU -

- Formula:

$$\begin{cases} x & \text{if } x > 0 \\ 0.01x & \text{if } x \leq 0 \end{cases}$$

- Pros: Fixes dying ReLU, allows small gradient for negatives
- Con: Needs tuning (slope)

6) ELU -

- Formula:

$$\begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases}$$

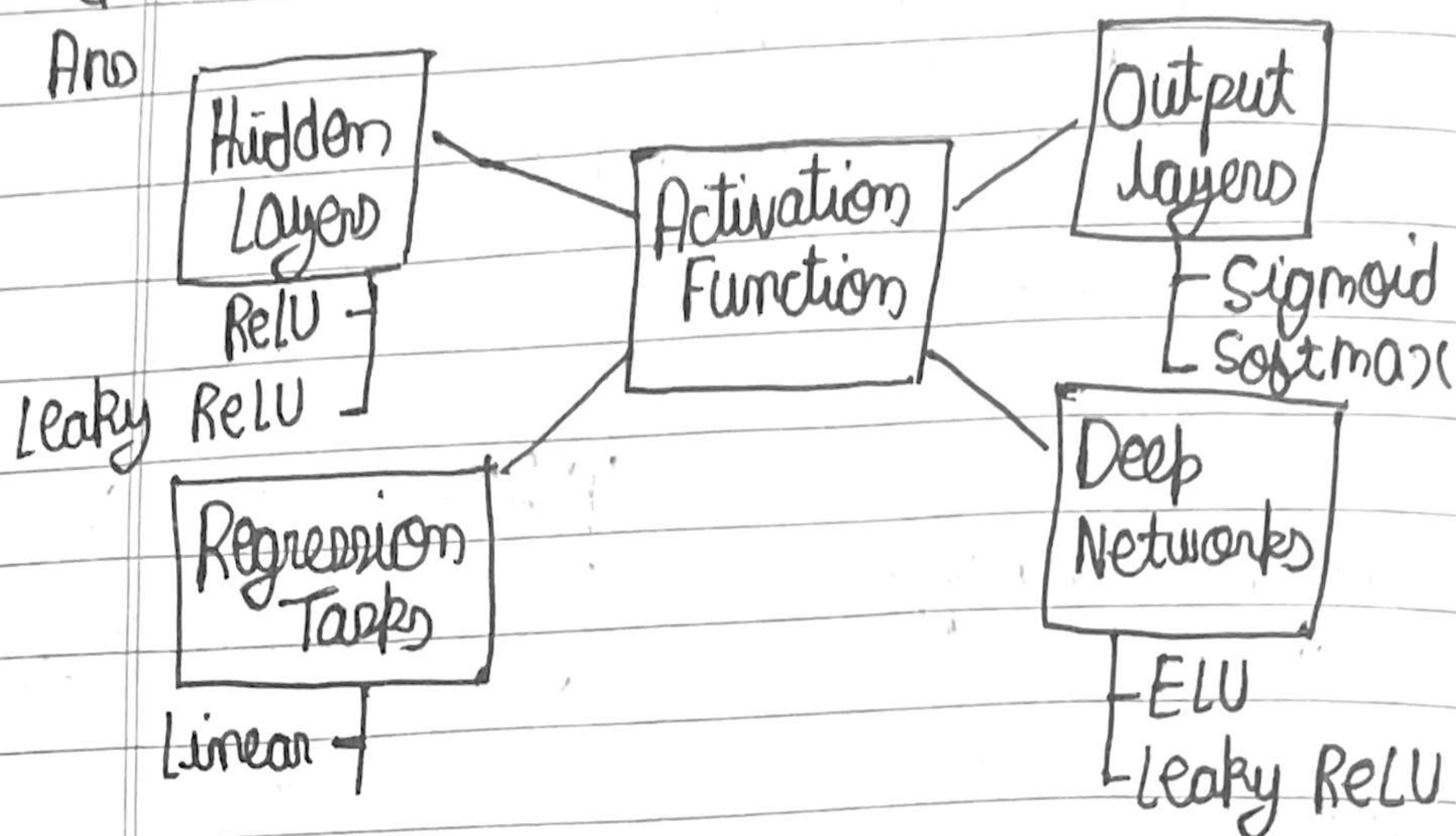
- Pros: Smooth gradients, faster learning, handles negative values
- Con: More computational cost

7) Softmax -

- Converts outputs $\rightarrow$ probabilities (sum = 1)
- Pros: Interpretable outputs
- Con: Sensitive to large values costly
- Used in multi class classification

Q How to choose the right activation function?

Ans



Loss Functions -

- Measures error between predicted ($\hat{y}$) and actual (y)
- Guides training via gradient descent
- Affects model performance & learning behaviour

Regression Loss -

1) MSE (Mean Squared Error) -

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

- Penalizes large errors
- Sensitive to outliers

2) MAE (Mean Absolute Error) -

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

- Robust to outliers
- Not differentiable at 0

3) Huber Loss -

$$L = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & |y - \hat{y}| \leq \delta \\ \delta |y - \hat{y}| - \frac{1}{2}\delta^2 & |y - \hat{y}| > \delta \end{cases}$$

- Combined MSE + MAE
- Robust + smooth

Imbalanced data $\rightarrow$ Focal Loss

Outliers $\rightarrow$ MAE/Huber

Better convergence $\rightarrow$ Smooth Losses

Always match loss with task + model

Date |

Page |

Classification Loss -

1) Binary Cross Entropy (Log Loss) -

$$BCE = -\frac{1}{n} \sum [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

- For binary classification

- Uses probabilities

2) Categorical Cross Entropy -

$$CCE = -\sum \sum y_{ij} \log(\hat{y}_{ij})$$

- For multi class classification

4) Hinge Loss (Sum) -

$$L = \frac{1}{n} \sum \max(0, 1 - y_i \cdot \hat{y}_i)$$

- Margin based classification

3) KL Divergence -

$$D_{KL}(P||Q) = \sum P(x) \log \frac{P(x)}{Q(x)}$$

- Measures difference between distributions

Choosing Loss functions -

1) Regression $\rightarrow$ MSE, MAE

2) Classification $\rightarrow$ Cross-Entropy

3) Based on output type (continuous vs probability)

Backpropagation in Neural Network -

- Backward propagation of errors to minimize loss

- Uses chain rule + gradient descent

- Updates weights (w) & biases (b)

- Enables learning in multi layer networks

- Advantages:

- Efficient weight updates & using gradients

- Scalable to deep networks

- Automates learning process

Steps:

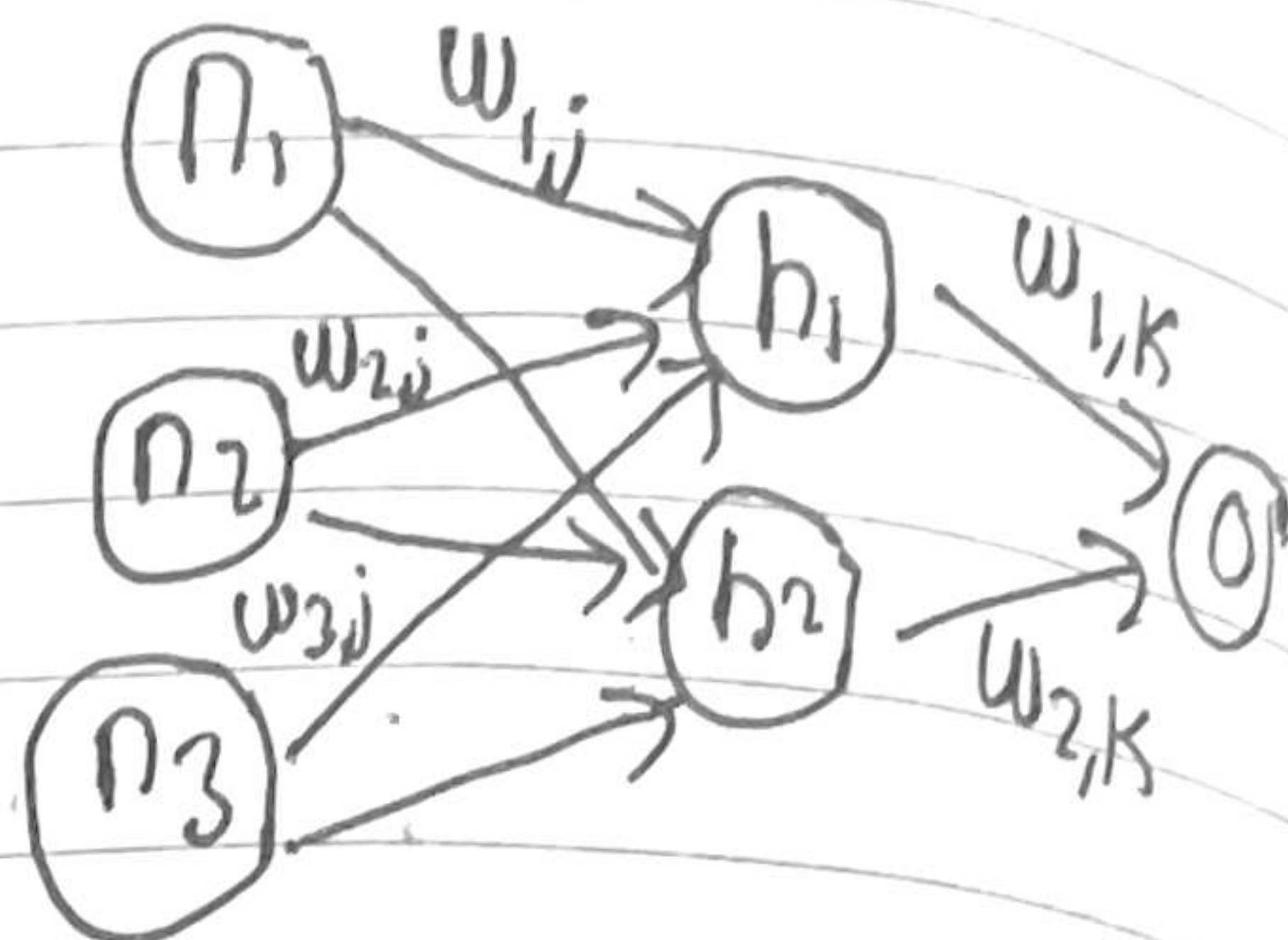
(i) Forward Pass

Compute weighted sum: $z = \sum w_i x_i + b$

Apply

activation: $a = f(z)$

Pass output layer by layer



(ii) Loss calculation

eg (MSE) $E = \frac{1}{2} (t - y)^2$

(iii) Backward Pass

Compute gradients using chain rule

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

(iv) Weight Update

Gradient descent update rule:

$$w_{\text{new}} = w - \eta \frac{\partial E}{\partial w}$$

η = learning rate

— Error propagates output $\rightarrow$ hidden $\rightarrow$ input

— Activation function derivative is crucial

— Repeated over epochs $\rightarrow$ loss decreases

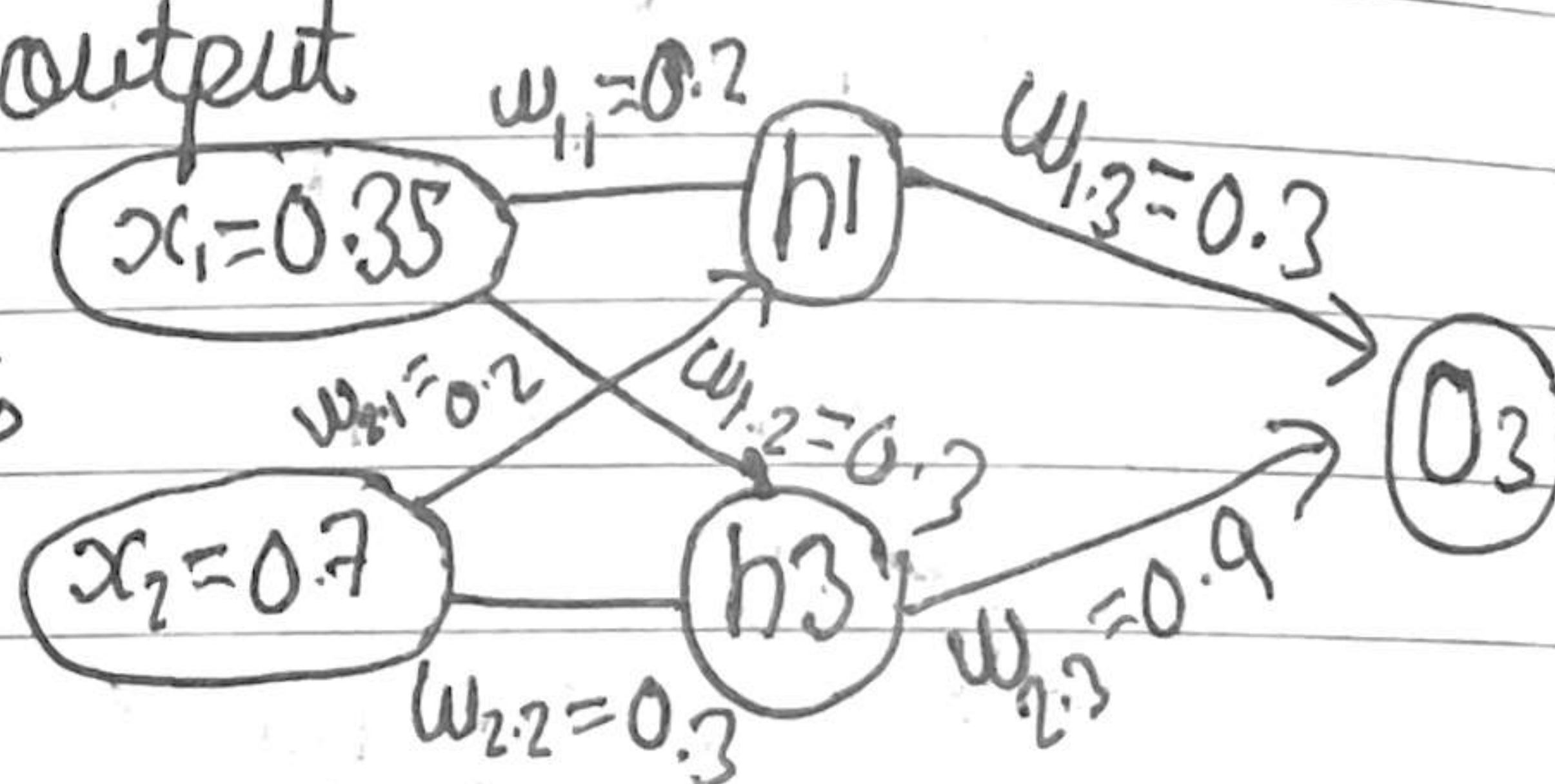
— Flow:

Forward pass $\rightarrow$ Compute output

Compute loss $\rightarrow$ Error

Backward pass $\rightarrow$ Gradients

Update weights $\rightarrow$ Repeat



— eg-

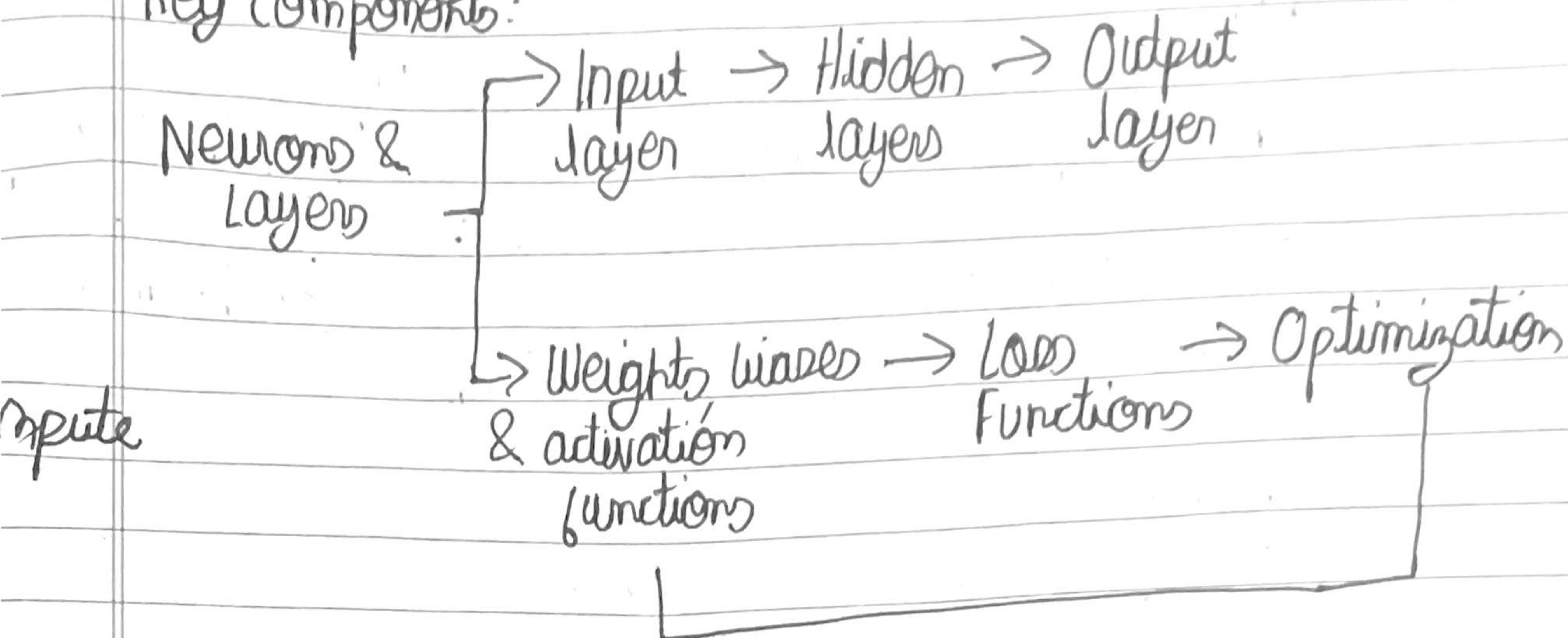
Inputs $\rightarrow$ Hidden layers (h_1, h_2) $\rightarrow$ Output

Each layer: $h = f(\sum w_i x_i + b)$

Final output used to compute error $\rightarrow$ backpropagate $\rightarrow$ update weights

Deep Learning -

- Uses multi-layered neural networks
- Learns from large datasets to make predictions automatically
- Key components:



- Types:

- Artificial Neural Networks (ANN)
- Convolutional Neural Networks (CNN)
- Recurrent Neural Networks (RNNs)
- Long Short Term Memory Networks (LSTMs)
- Generative Adversarial Networks (GAN)
- Autoencoders

- Challenges:

- Requires large datasets for training
- Needs high performance hardware
- Risks of overfitting to training data
- Lack of transparency & interpretability

Neural Networks -

- Inspired by human brain
- Composed of layers of neurons (nodes)
- Process: input $\rightarrow$ weighted computation $\rightarrow$ Activation $\rightarrow$ Output

- Components:

- Layers (Input, hidden, output)
- Weights & biases
- Forward Propagation
- Activation Functions
- Loss Functions
- Learning Rate
- Optimization

- Advantages:

- High accuracy for complex tasks
- Automatic feature extraction
- End to end mapping

- Disadvantages:

- Requires large datasets
- Complex training
- Risk of overfitting

- Applications:

- Self driving cars
- Medical diagnosis
- Speech recognition
- Recommendation systems

- CNN (Convolutional Neural Networks):

Basics

- Type of deep neural network
- Inspired by visual cortex
- Detects patterns like edges, shapes, textures

Key characteristics

- Local feature detection
- Learns spatial hierarchies
- Works on images, videos, audio, text

Important points

- Developed/organized by Yann LeCun
- Used for pattern recognition in structured arrays
- Multiple CNN architectures exist